

8

Algoritmos — del pseudocódigo al diagrama de flujo

Guía 13-8-TIC

LO QUE TE LLEVAS HOY

1 algoritmo cotidiano (preparar arepa, llegar al colegio, decidir qué ropa usar, hacer cabello, lavar tenis) escrito en pseudocódigo claro de al menos 8 pasos con uso de variables y al menos 1 if-else + traducción del mismo algoritmo a diagrama de flujo usando los 4 símbolos correctos (óvalo inicio/fin, rectángulo proceso, rombo decisión, paralelogramo entrada/salida) + breve descripción de 5 líneas explicando cómo se conectan los dos formatos

ESTA GUÍA ES DE

Nombre completo

Grupo

Fecha

Apertura: Algoritmos — del pseudocódigo al diagrama de flujo

Saber ancestral

En las cocinas del Valle del Cauca y en las casas de los abuelos del Pacífico, las recetas no se escribían: se transmitían. Pero cualquier abuela que enseñaba a hacer arepa o sancocho seguía un orden inquebrantable que cualquier nieto reconoce: **primero esto, después aquello, si pasa esto otra cosa**. “*Primero pelas el plátano. Después lo cortas en rodajas. Si está verde, lo fríes. Si está maduro, lo asas. Si quedan pedazos, los muelo*”. La receta tiene 4 piezas silenciosas que cualquier algoritmo profesional reconocería: (1) **Secuencia**: el orden de los pasos importa (no se puede cortar antes de pelar). (2) **Decisión**: a veces hay dos caminos según una condición (verde → freír / maduro → asar). (3) **Repetición**: a veces hay que hacer lo mismo varias veces (“*revuelve hasta que esté liso*”). (4) **Entrada y salida**: la materia prima entra (plátano), el plato sale (frito o asado). La sabiduría de la receta ancestral es la **forma natural del algoritmo**: la programación moderna no inventó esta estructura, solo le dio nombres formales.

Saber contemporáneo

Un **algoritmo** es una secuencia ordenada de pasos finitos que resuelve un problema. Se puede expresar de varias formas: (1) **Pseudocódigo**: lenguaje cercano al humano pero estructurado, con instrucciones claras. Ejemplo: LEER edad; SI edad >= 18 ENTONCES MOSTRAR ``mayor``; SINO MOSTRAR ``menor``. (2) **Diagrama de flujo**: representación visual con cajas y flechas, usando 4 símbolos estándar: **óvalo** para inicio/fin, **rectángulo** para procesos, **rombo** para decisiones, **paralelogramo** para entrada/salida de datos. Las flechas indican el orden. La regla profesional del pensamiento computacional: **antes de programar, escribir el algoritmo en pseudocódigo o diagrama**. Saltarse esta etapa lleva a código desordenado y errores difíciles de depurar. Los dos formatos son complementarios: el pseudocódigo es preciso, el diagrama es visual; juntos comunican la lógica a personas técnicas y no técnicas.

Pregunta-puente

¿Qué sabía la abuela al transmitir la receta del sancocho con orden estricto, que el programador novato olvida cuando salta a escribir código sin pensar el algoritmo? ¿Y por qué un diagrama de flujo bien hecho permite explicar un algoritmo a alguien que no programa?

Saber y saber hacer

Al terminar podrás: (1) **identificar** las 4 estructuras básicas de un algoritmo (secuencia, decisión, repetición, entrada/salida) en una receta o tarea cotidiana; (2) **explicar** la diferencia entre pseudocódigo y diagrama de flujo y cuándo conviene cada uno; (3) **crear** un algoritmo en pseudocódigo y traducirlo a diagrama de flujo con los 4 símbolos correctos; (4) **evaluar** un algoritmo ajeno detectando si los pasos están ordenados, si las decisiones cubren todos los casos y si los símbolos del diagrama son correctos.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Pensamiento computacional aplicado a sistemas de control con sensores y actuadores (MEN, T&I)
Referentes	Pensamiento computacional · MakeCode / micro:bit · Logos como razón universal
Producto final	1 algoritmo cotidiano (preparar arepa, llegar al colegio, decidir qué ropa usar, hacer cabello, lavar tenis) escrito en pseudocódigo claro de al menos 8 pasos con uso de variables y al menos 1 if-else + traducción del mismo algoritmo a diagrama de flujo usando los 4 símbolos correctos (óvalo inicio/fin, rectángulo proceso, rombo decisión, paralelogramo entrada/salida) + breve descripción de 5 líneas explicando cómo se conectan los dos formatos

→ Antes de empezar, mira el viaje completo. En las sesiones 1-2 aprendiste a decidir con AND/OR/NOT y a verificar con tablas de verdad. Hoy aprendes a escribir y dibujar la lógica completa en formato profesional. Las próximas sesiones (sensores, actuadores, micro:bit) tomarán tus pseudocódigos como punto de partida para programar.

Ruta MILC: así vamos a aprender

1

Escucha
Observo, escucho
y pregunto.

2

Entender
Sistematizo
y ordeno.

3

Hacer
Construyo y aplico.

4

Cierre
Evalúo y reflexiono.

→ Antes de teorizar, vas a hacer un ejercicio del abuelo: piensa en algo que sepas hacer bien y enséñalo por escrito con orden de pasos (la receta de algo, la rutina de la mañana, cómo preparar el morral). Esa transmisión escrita ya es algoritmo en lenguaje natural.

Estación 1 de 3 · Escucha

Escena para escuchar

Actividad 1 · IDENTIFICA — Ejercicio del abuelo (15 min · individual). Escribe en una hoja, paso a paso, cómo se hace algo que dominas: preparar un sándwich, alistar el morral del día siguiente, hacer un nudo de corbata, lavar los tenis. Reglas: (1) numera los pasos en orden. (2) usa frases cortas y claras. (3) si en algún paso hay una decisión (“si está sucio entonces lavar; si no, guardar”), marca esa rama. (4) no te saltes ningún paso por obvio. La regla: si un extraterrestre leyera tu lista, debería poder ejecutarla.

MARCA CUANDO LO HAYAS HECHO

- ☐ Escribí los pasos en orden numerado.
- ☐ Identifiqué al menos 1 punto de decisión.

☐ Mi lista podría ser ejecutada por alguien sin contexto.

Lo que va al cuaderno (Actividad 1 · IDENTIFICA). Título: «Actividad 1 — Receta del abuelo». **Formato:** pasos numerados (al menos 8) + decisión marcada. **Sabes que terminaste cuando** otro estudiante podría ejecutar tu lista paso a paso.

→ Ya tienes el ejercicio. Ahora vas a entender la **anatomía formal**: las 4 estructuras básicas, el pseudocódigo con convenciones, los 4 símbolos del diagrama de flujo, los 5 errores típicos del aprendizaje que delatan falta de pensamiento previo.

Estación 2 de 3 · Entender

Lo que vas a entender

Tu lista del abuelo ya es algoritmo en lenguaje natural. Ahora necesitas la **anatomía formal** del oficio computacional: las 4 estructuras, las convenciones del pseudocódigo, los 4 símbolos del diagrama de flujo.

- 1 Las 4 estructuras básicas se descomponen así: (1) **Secuencia**: pasos en orden, uno tras otro (PASO 1; PASO 2; PASO 3). (2) **Decisión** (selección): SI condición ENTONCES PASO_A SINO PASO_B (sesión 1). (3) **Repetición** (iteración): REPETIR n VECES o MIENTRAS condición para hacer lo mismo varias veces. (4) **Entrada/Salida**: LEER variable para recibir datos del usuario, MOSTRAR variable para devolver resultados. Estas 4 estructuras combinadas pueden expresar cualquier algoritmo posible.
- 2 Las convenciones del **pseudocódigo** son simples: (a) palabras clave en MAYÚSCULA (SI, ENTONCES, SINO, MIENTRAS, REPETIR, LEER, MOSTRAR). (b) variables en minúscula descriptiva (edad, nombre, temperatura). (c) indentación para mostrar jerarquía (lo que está dentro de un SI va indentado). (d) un paso por línea. (e) usar punto y coma o cambio de línea para separar pasos. No hay sintaxis estricta: lo importante es que sea claro.
- 3 Los **4 símbolos** del diagrama de flujo son universales: (1) **Óvalo**: inicio y fin del algoritmo. Siempre 1 inicio y al menos 1 fin. (2) **Rectángulo**: proceso (cálculo, asignación, paso de acción). (3) **Rombo**: decisión. Tiene 1 entrada y 2 salidas (sí/no, verdadero/falso). (4) **Paralelogramo**: entrada o salida de datos. Las flechas conectan los símbolos en orden. Un diagrama bien hecho se lee de arriba abajo (o izquierda a derecha) y no tiene flechas cruzadas innecesariamente.

4

Algoritmo para escribir un algoritmo: (1) define la tarea en 1 frase. (2) identifica entrada (datos que necesitas) y salida (resultado esperado). (3) lista pasos en orden. (4) identifica decisiones (puntos donde el algoritmo elige camino). (5) escribe en pseudocódigo. (6) traduce a diagrama. (7) prueba mentalmente con un caso de entrada y verifica que llegas al resultado.

Anatomía del algoritmo

4 estructuras: secuencia / decisión / repetición / E/S.

Pseudocódigo: palabras clave + indentación.

4 símbolos: óvalo / rectángulo / rombo / paralelogramo.

Flujo: arriba-abajo o izquierda-derecha.

1 inicio, 1+ fin.

Errores comunes y su corrección

(1) **Saltarse pasos obvios:** el algoritmo asume que el ejecutor sabe lo que no se dijo. (2)

Mezclar símbolos en diagrama: usar rectángulo donde debe ir rombo, o viceversa. (3)

Diagrama sin inicio o sin fin: confunde al lector. (4) **Decisiones sin todas las ramas:** rombo con solo una salida deja al algoritmo sin camino. (5) **Flechas cruzadas sin necesidad:** produce diagramas ilegibles; conviene reorganizar.

Lo que va al cuaderno (Actividad 2 · EXPLICA). Título: «Actividad 2 — Anatomía del algoritmo».

Formato: (a) las 4 estructuras con ejemplo; (b) los 4 símbolos dibujados con su uso; (c) los 5 errores con marca del que más temes. **Sabes que terminaste cuando** podrías dibujar un diagrama desde cero sin abrir esta guía.

→ Tienes el método. Toca aplicarlo: eliges 1 tarea cotidiana, la escribes en pseudocódigo claro (8+ pasos, variables, 1+ if-else), la traduces a diagrama de flujo con los 4 símbolos, y escribes 5 líneas explicando cómo se conectan los dos formatos.

Estación 3 de 3 · Hacer

Cómo vas a construir tu producto

Actividad 3 · CREA + EVALÚA — Algoritmo en pseudocódigo + diagrama de flujo (40 min · individual). Hora de crear. Eliges 1 tarea cotidiana, la escribes en pseudocódigo de 8+ pasos con variables y al menos 1 if-else, y la traduces a diagrama de flujo con los 4 símbolos correctos.

- 1 Tu algoritmo se construye en 5 pasos: (1) elegir tarea concreta (preparar arepa, decidir ropa según clima, hacer la maleta del fin de semana, planificar la ruta al colegio). (2) identificar entrada (qué necesitas saber) y salida (qué se entrega). (3) escribir pseudocódigo de 8+ pasos con uso de variables (clima, tiempo_disponible, etc.) y al menos 1 if-else. (4) traducir a diagrama de flujo con los 4 símbolos. (5) escribir 5 líneas explicando cómo se conectan los dos formatos (qué se ve mejor en cada uno).
- 2 Sigue el patrón “**pseudocódigo primero, diagrama después**”: el pseudocódigo te obliga a pensar la lógica; el diagrama te obliga a verificar el flujo visual. Si haces el diagrama primero, las cajas tienden a ser vagas. Si haces el pseudocódigo primero, el diagrama queda preciso.
- 3 Las herramientas para hacer el diagrama: (a) papel y lápiz (lo más rápido para borrador). (b) **draw.io** (gratis, en línea, símbolos profesionales). (c) Google Slides o PowerPoint con formas básicas. (d) Lucidchart (gratis con cuenta). Para esta sesión es válido cualquiera; lo importante es que los símbolos sean los 4 estándar.
- 4 Algoritmo: (1) elegir tarea. (2) escribir pseudocódigo. (3) verificar con caso de entrada. (4) hacer borrador del diagrama. (5) digitalizar el diagrama (opcional pero recomendado). (6) escribir las 5 líneas de conexión. (7) revisar con un compañero (¿entiende el algoritmo solo viendo el diagrama?).

Checklist antes de entregar

- ☐ Tarea elegida concreta y cotidiana.
- ☐ Pseudocódigo con 8+ pasos.
- ☐ Al menos 1 variable usada.
- ☐ Al menos 1 if-else.
- ☐ Diagrama de flujo con los 4 símbolos.
- ☐ 1 inicio (óvalo) y al menos 1 fin (óvalo).
- ☐ Sin flechas cruzadas innecesariamente.
- ☐ 5 líneas explicando la conexión entre formatos.

Plantilla de trabajo

Tarea. *Una sola, concreta, cotidiana.*

Entrada. *Datos que recibo.*

Salida. *Resultado que produzco.*

Pseudocódigo. *8+ pasos, variables, 1+ if-else.*

Diagrama. *Con los 4 símbolos.*

Conexión. *Cómo se complementan los formatos.*

Lo que va al cuaderno (Actividad 3 · CREA + EVALÚA). Título: «Actividad 3 — Algoritmo + diagrama».

Formato: (a) pseudocódigo con sus 8+ pasos; (b) diagrama de flujo dibujado o pegado; (c) 5 líneas de conexión. **Sabes que terminaste cuando** podrías delegar la ejecución del algoritmo a alguien con solo tu diagrama.

→ *Lo que sigue resume qué entregas hoy: 1 algoritmo en pseudocódigo + 1 diagrama de flujo + descripción de conexión. El criterio principal: que un compañero pueda ejecutar mentalmente tu algoritmo y llegar al resultado correcto, mirando solo el diagrama.*

Producto de la sesión

Algoritmo en pseudocódigo + diagrama de flujo + descripción de conexión.

Criterios de calidad

(1) Pseudocódigo con 8+ pasos claros. (2) Uso de al menos 1 variable. (3) Al menos 1 if-else. (4) Diagrama de flujo con los 4 símbolos correctos. (5) Flujo legible (arriba-abajo o izquierda-derecha). (6) Descripción de cómo se complementan los formatos.

→ *Antes de cerrar, mira el algoritmo desde las cinco dimensiones humanas. Escribir el pseudocódigo antes de programar es respeto por el oficio; saltar a programar es desperdicio que produce errores caros.*

Evaluación liberadora · cinco dimensiones**Sentido de la evaluación**

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

→ Y para terminar, tres voces sobre la planeación del oficio. Dussel sobre los algoritmos que liberan o atrapan a sus usuarios, Epicteto sobre la disciplina de planear antes de actuar, Floridi sobre el algoritmo claro como ética del oficio digital.

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

Un algoritmo claro libera al ejecutor; un algoritmo opaco lo somete a la voluntad oculta del programador.

— formulación de esta guía, al modo de Enrique Dussel. No es una cita textual.

Cuando tu algoritmo está escrito con claridad (pseudocódigo legible + diagrama explícito), cualquiera puede revisarlo, ajustarlo, criticarlo. La filosofía de la liberación pide esa transparencia. Cuando el algoritmo es opaco, solo el programador lo entiende y el ejecutor depende de él. La phronesis del oficio consiste en **escribir algoritmos que liberen, no que aten.**

¿Mi algoritmo es lo suficientemente claro para que otro lo modifique sin pedirme ayuda?

Estoicismo · Epicteto · cuidado interior

Planear antes de actuar es virtud del oficio; improvisar es debilidad disfrazada de espontaneidad.

— formulación de esta guía, al modo de Epicteto. No es una cita textual.

Es tentador saltar a programar sin escribir pseudocódigo. La disciplina estoica recomienda lo opuesto: *planea antes de actuar*. Escribir el algoritmo en pseudocódigo y diagrama te ahorra horas de debug posterior. La phronesis del oficio honra al programador que dedica 15 minutos a planear antes de escribir 2 horas de código.

¿Estoy planeando con pseudocódigo antes de actuar, o salto directo al código sin pensar?

Floridi · lente de la infoesfera

El algoritmo claro y revisable es la nueva ética del oficio digital en la era automatizada.

— formulación de esta guía, al modo de Luciano Floridi. No es una cita textual.

En la era de la automatización, los algoritmos deciden por nosotros (filtros de redes, recomendaciones, créditos). La ética digital exige que esos algoritmos sean revisables: cualquier persona técnica debe poder leer el pseudocódigo o ver el diagrama y entender cómo se decide. Tu hoja de hoy te entrena en esa práctica que rige toda profesión técnica del siglo XXI.

¿Mi pseudocódigo permite revisar la lógica, o oculta la decisión bajo código complejo?

Nota para el docente. Las tres formulaciones son del autor de la guía, no citas textuales de los pensadores: recogen su lente para pensar el tema, no sus palabras. Si vas a citarlos en clase, remite a la obra original.

Mi autoevaluación · marca una casilla por fila

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	☐ Explico las ideas con mis palabras.	☐ Reconozco las ideas, falta precisión.	☐ Necesito repasar lo básico.
Pensamiento computacional	☐ Usé los pilares al armar mi producto.	☐ Usé algunos pilares.	☐ Necesito releer la estación 2.
Aplicación y producto	☐ Mi producto cumple los criterios.	☐ Está armado, con ajustes pendientes.	☐ Aún está en construcción.
Reflexión 5 dimensiones	☐ Respondí cada una con honestidad.	☐ Respondí algunas con detalle.	☐ Mis respuestas son muy generales.
Triángulo de pensamiento	☐ Conecté las 3 voces con mi trabajo.	☐ Conecté al menos una.	☐ No las relaciono con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”
Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo: _____

Fecha: _____ **Curso/grupo:** _____

Mi firma: _____

Para la próxima sesión. Llega con tu algoritmo (pseudocódigo + diagrama). En la sesión 4 vas a programar tu primer programa real en **MakeCode** para el micro:bit, usando sensores que perciben el mundo. Los algoritmos en papel de hoy son la base; el código que reacciona al entorno de mañana se monta sobre esa planeación.